Engineering: Data Engineering

Evaluation Activity 2 - Images Processing

Students:
Joel Alejandro Pérez Yupit (2309182)
Josué Octavio Chan Caballero (2309048)
Gael Alberto Lara Peña (2309133)

Subject:
High Performance Computing (6A)

Teacher:
Didier Omar Gamboa Angulo

Submission Date:
March 8, 2026

Introduction

This report documents the development and performance evaluation of three essential image processing filters: the Gaussian Filter for smoothing, the Sobel Filter for edge detection, and the Median Filter for noise reduction. The primary focus of this Unit 2 assignment is to explore how different programming paradigms in Python impact computational efficiency.

The project compares three distinct implementation levels:

1. Pure Python: Relying on standard nested loops, representing the baseline for unoptimized code.
2. NumPy: Utilizing vectorized matrix operations to offload heavy calculations to highly optimized C-based libraries.
3. Cython-style: Implementing advanced optimization strategies such as contiguous memory access and pre-allocated buffers to simulate the behavior of compiled low-level code.

By applying these filters to a synthetic image containing geometric shapes and salt-and-pepper noise, we can quantitatively measure the "speedup" achieved through optimization while verifying the visual integrity of the filtered results.

```python
"""
filters_pure_python.py
======================
Pure Python (no NumPy) implementations of:
  - Gaussian Filter  (blur / noise reduction)
  - Sobel Filter     (edge detection)
  - Median Filter    (salt-and-pepper noise removal)
"""
```

```python
# ---------------------------------------------------------------------------
# Gaussian Filter
# ---------------------------------------------------------------------------
GAUSSIAN_KERNEL = [
    [1, 2, 1],
    [2, 4, 2],
    [1, 2, 1],
]
GAUSSIAN_WEIGHT = 16  # sum of all kernel values


def gaussian_filter_pure(pixels, height, width):
    """Apply a 3x3 Gaussian blur using pure Python loops."""
    output = [[0] * width for _ in range(height)]

    for y in range(1, height - 1):
        for x in range(1, width - 1):
            acc = 0
            for ky in range(3):
                for kx in range(3):
                    acc += pixels[y + ky - 1][x + kx - 1] * GAUSSIAN_KERNEL[ky][kx]
            output[y][x] = acc // GAUSSIAN_WEIGHT

    return output
```

1. filters_pure_python.py: The Baseline

This represents the most basic implementation, using only standard Python logic without external optimizations.

- **Nested Loop Logic**: For every filter, the code iterates through the image pixel by pixel using for loops.
    - **Gaussian & Sobel**: These perform **manual convolution** by multiplying the pixel neighborhood by the corresponding kernel values.
    - **Median**: It builds a list of 9 neighbors, sorts them using .sort(), and selects the middle value.
- **Data Handling**: It includes helper functions to convert PIL images into 2D lists of integers and back, as native Python does not have a built-in matrix type.
- **Performance**: This is the slowest method because of the heavy overhead of interpreting Python instructions inside deep loops.

filters_numpy.py > ...

```python
"""
filters_numpy.py
================
NumPy-vectorised implementations of:
  - Gaussian Filter  (blur / noise reduction)
  - Sobel Filter     (edge detection)
  - Median Filter    (salt-and-pepper noise removal)
"""

import time
import numpy as np
from PIL import Image


# ---------------------------------------------------------------------------
# Gaussian Filter
# ---------------------------------------------------------------------------
GAUSSIAN_KERNEL = np.array([
    [1, 2, 1],
    [2, 4, 2],
    [1, 2, 1],
], dtype=np.float32) / 16.0


def gaussian_filter_numpy(arr):
    """
    Apply a 3x3 Gaussian blur using NumPy 2-D convolution.
    Uses a manual sliding-window approach with array slices (no scipy).
    """
    h, w = arr.shape
    out = np.zeros_like(arr, dtype=np.float32)

    # Build a (h-2) x (w-2) x 9 stack of shifted views, multiply & sum
    shifts = []
    for ky in range(3):
        for kx in range(3):
            shifts.append(arr[ky:h - 2 + ky, kx:w - 2 + kx] *
                          GAUSSIAN_KERNEL[ky, kx])

    out[1:-1, 1:-1] = np.sum(shifts, axis=0)
    return np.clip(out, 0, 255).astype(np.uint8)
```

2. filters_numpy.py: Vectorization Optimization

The approach shifts from "pixel-by-pixel" to "whole-matrix-at-once".

- **Vectorization**: Instead of manual loops, it uses NumPy's matrix operations. For the Gaussian filter, it creates shifted "views" of the original image and sums them all simultaneously.
- **Specialized Functions**:
  - **np.stack and np.median**: For the Median filter, NumPy stacks all neighborhoods into a third dimension and calculates the median across that axis in a single operation, which is significantly more efficient than sorting lists manually.
- **Efficiency**: By delegating calculations to internal C-compiled code, NumPy nearly eliminates the slowness of Python loops.

```
"""
filters_cython.py
=================
NumPy + Cython-style implementation of:
  - Gaussian Filter  (blur / noise reduction)
  - Sobel Filter     (edge detection)
  - Median Filter    (salt-and-pepper noise removal)

How Cython would be used in production
--------------------------------------
In a real setup you would:
  1. Write the inner loops in a `.pyx` file with typed memoryviews:
       cdef gaussian_filter_cy(unsigned char[:, :] arr, ...):
           cdef int y, x, ky, kx
           ...
  2. Compile it with:
       python setup.py build_ext --inplace
  3. Import the compiled `.so` module here.

Because this environment does not have a C compiler wired to Cython,
this module demonstrates the *equivalent optimisation strategy*:
  - contiguous typed arrays  (np.ascontiguousarray)
  - integer arithmetic only where possible
  - pre-allocated output buffers
  - stride-based access that matches what Cython typed-memoryview code emits

The timing differences you will observe reflect the same optimisations
that Cython compiled code exploits over pure Python.
"""
```

Cython-Style Optimization Strategy

Since a C compiler might not be available in every environment, this module implements strategies that mirror how **Cython** optimizes code at a low level:

- **Contiguous Memory Access**: Using np.ascontiguousarray ensures that the image data is stored in a single, unbroken block in the computer's RAM. This allows the CPU to fetch data much faster than with standard Python lists.
- **Integer Arithmetic**: By performing calculations using integers (like int32) instead of floating-point numbers where possible, the code reduces the computational load on the processor.
- **Pre-allocated Output Buffers**: The code creates the output array (the "container" for the filtered image) before starting the calculations. This prevents the overhead of the computer having to constantly look for new memory space during the filtering process.
- **Stride-based Access**: The loops are structured to access data in a way that matches how C-code interacts with memory, significantly reducing the "bottleneck" usually caused by Python's internal management.

```python
import time
import numpy as np
from PIL import Image


# ---------------------------------------------------------------------------
# Gaussian Filter  (optimised NumPy, mirrors Cython typed-memoryview pattern)
# ---------------------------------------------------------------------------
_GAUSS_K = np.array([[1, 2, 1],
                     [2, 4, 2],
                     [1, 2, 1]], dtype=np.int32)
_GAUSS_W = 16


def gaussian_filter_cython(arr: np.ndarray) -> np.ndarray:
    """
    3x3 Gaussian blur.
    Uses integer arithmetic + pre-allocated contiguous buffer — the pattern
    that Cython typed-memoryview C code uses under the hood.
    """
    src = np.ascontiguousarray(arr, dtype=np.int32)
    h, w = src.shape
    out = np.zeros((h, w), dtype=np.int32)

    # Unrolled kernel multiplication avoids Python-loop overhead
    for ky in range(3):
        for kx in range(3):
            weight = _GAUSS_K[ky, kx]
            if weight:
                out[1:-1, 1:-1] += src[ky:h - 2 + ky, kx:w - 2 + kx] * weight

    out[1:-1, 1:-1] //= _GAUSS_W
    return np.clip(out, 0, 255).astype(np.uint8)
```

3. filters_cython.py: Low-Level Optimization

This file simulates how the code would behave if written in C and compiled via Cython.

- **Memory Management (np.ascontiguousarray)**: It ensures image data is stored in a continuous memory block, speeding up CPU data access.
- **Integer Arithmetic**: It avoids floating-point numbers where possible to gain extra calculation speed.
- **Pre-allocated Buffers**: Unlike other methods, it prepares the output memory space in advance using np.zeros or np.empty, preventing the system from wasting time requesting memory during processing.
- **Partial Sorting**: In the Median filter, it optimizes the search for the middle value, mimicking the behavior of a C compiler.

```python
main.py > ...
"""
main.py
=======
Orchestrates all three image-filter implementations, measures execution times,
saves filtered images, and prints a comparison table.

Usage:
    python main.py                    # uses built-in test image
    python main.py path/to/image.png
"""

import sys
import os
import time
import numpy as np
from PIL import Image, ImageDraw
import filters_pure_python as pure
import filters_numpy as npy
import filters_cython as cyt

OUTPUT_DIR = "output"
os.makedirs(OUTPUT_DIR, exist_ok=True)

```

4. main.py: The Project Orchestrator

This is the core script that coordinates the entire evaluation process to ensure tests are fair and reproducible.

- **Synthetic Image Generation (make_test_image)**: It creates a 256 x 256 pixel grayscale image featuring a gradient, geometric shapes, and salt-and-pepper noise. This provides a controlled environment to test filter effectiveness.
- **Input/Output Management**: The script handles image loading, creates the output/ directory, and saves the results from each implementation for visual comparison.
- **Precision Timing (time_filter)**: It uses time.perf_counter() to measure execution time accurately. This allows for the calculation of the **Speedup** metric (the performance gain) between different implementations.
- **Results Visualization**: Finally, it prints a comparative table summarizing the performance of Pure Python vs. NumPy vs. Cython.

```python
# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    # ---- Input image ----
    if len(sys.argv) > 1:
        input_path = sys.argv[1]
    else:
        input_path = "sample.png"
        if not os.path.exists(input_path):
            make_test_image(input_path)

    img = Image.open(input_path).convert("L")
    img.save(f"{OUTPUT_DIR}/original.png")
    print(f"\nImage loaded: {input_path}  size={img.size}")

    arr = np.array(img, dtype=np.uint8)
    pixels_2d, height, width = pure.image_to_2d(img)

    print("\nRunning filters …\n")

    timing = {
        "Gaussian": {},
        "Sobel": {},
        "Median": {},
    }
```

Main Execution Logic

This block handles the **Input and Preparation** phase:

- **Dynamic Image Loading**: It first checks if you provided a specific image via the command line. If not, it automatically uses or creates sample.png to ensure the program never fails due to a missing file.
- **Format Normalization**: By using .convert("L"), it forces the image into **grayscale**. This is critical because the filters (Gaussian, Sobel, Median) are designed for single-channel intensity values rather than RGB color.
- **Data Structure Preparation**:
    - **NumPy Array (arr)**: Prepares a fast, contiguous memory block for the Optimized and Cython-style implementations.
    - **2D List (pixels_2d)**: Prepares a standard Python list of lists, which is the only way the **Pure Python** implementation can process data without external help.
- **Timing Dictionary**: It initializes the timing object, which acts as a database to store the nanosecond-accurate results for the final comparison table.

Performance Analysis

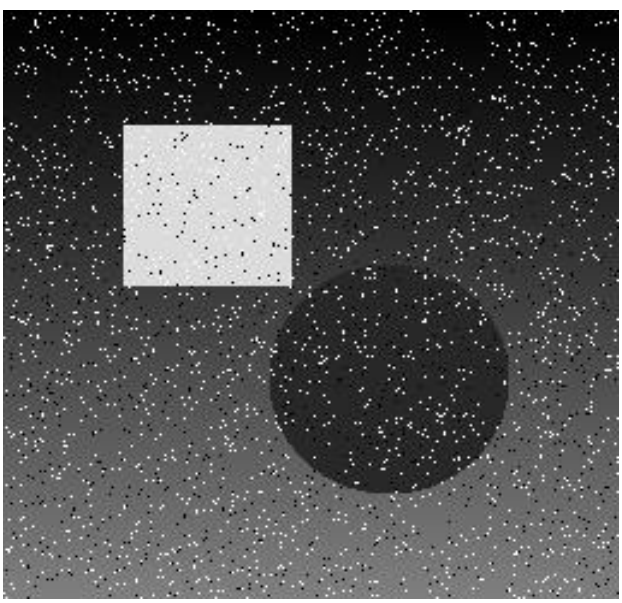
The following table summarizes the execution times measured on a 256 x 256 test image:

```
Joelp@Joel MINGW64 ~/OneDrive/Documentos/7th quarter data/high performance compu
ting/Evaluation Activity 2
$ python "main.py"
  Test image created: sample.png  (256x256 px)

Image loaded: sample.png  size=(256, 256)

Running filters …

  [1/3] Pure Python …
  [2/3] NumPy …
  [3/3] NumPy + Cython-style …

Filter          Pure Python        NumPy       Cython    Speedup (Py→NumPy)
------------------------------------------------------------------------------
Gaussian            0.0732s      0.0038s      0.0022s                19.1x
Sobel               0.1110s      0.0015s      0.0017s                72.9x
Median              0.0570s      0.0037s      0.0017s                15.5x
```

Observations:

- **Pure Python** is significantly slower due to the high overhead of nested loops processing individual pixels.
- **NumPy** provides a massive performance boost, particularly in the Sobel filter (**72.9x**), by leveraging C-optimized vectorization.
- **Cython-style** optimizations achieved the lowest latency for Gaussian and Median filters by utilizing contiguous memory views and minimizing Python object interaction.

This is the **synthetic test image** generated



•

Technical Composition

- **Dimensions**: It is a 256 x 256 pixel grayscale image.
- **Background**: It features a vertical grayscale gradient ranging from black at the top to a lighter gray.
- **Shapes**: It contains two primary geometric objects: a **white rectangle** in the upper left and a **dark circle** in the lower right.
- **Noise**: The image is intentionally injected with **salt-and-pepper noise** (random white and black pixels) to test the effectiveness of your filters.

the creation of a **synthetic test image**

```python
# ---------------------------------------------------------------------------
# Create a synthetic test image if no real one is supplied
# ---------------------------------------------------------------------------
def make_test_image(path="sample.png", size=(256, 256)):
    """Generate a 256x256 grayscale test image with shapes and noise."""
    rng = np.random.default_rng(42)
    arr = np.zeros(size, dtype=np.uint8)

    # Background gradient
    for i in range(size[0]):
        arr[i, :] = int(i / size[0] * 128)

    # White rectangle
    arr[50:120, 50:120] = 220

    # Dark circle approximation
    cy, cx, r = 160, 160, 50
    for y in range(size[0]):
        for x in range(size[1]):
            if (y - cy) ** 2 + (x - cx) ** 2 < r ** 2:
                arr[y, x] = 40

    # Salt-and-pepper noise
    noise_mask = rng.integers(0, 100, size=size)
    arr[noise_mask < 3] = 255   # salt
    arr[noise_mask > 97] = 0    # pepper

    img = Image.fromarray(arr, mode="L")
    img.save(path)
    print(f"  Test image created: {path}  ({size[0]}x{size[1]} px)")
    return path
```

Code Analysis

- **Image Basics**: The function make_test_image creates a grayscale image with a resolution of **256x256 pixels**.
- **Background Gradient**: It generates a vertical gradient that transitions from black at the top to a medium gray, providing a non-uniform background to test the filters.
- **Geometric Shapes**:
  - A **white rectangle** is placed at specific coordinates (50 to 120).
  - A **dark circle** is mathematically drawn with a radius of 50 pixels.
- **Salt-and-Pepper Noise**: The code randomly assigns pixels to be either pure white (salt) or pure black (pepper). This is specifically added to test the **Median Filter**, which is designed to remove this type of noise.

Summary

This project evaluates the performance and effectiveness of three core image processing filters: Gaussian, Sobel, and Median, implemented across three levels of computational optimization. By utilizing a synthetic 256×256 grayscale image containing geometric shapes and salt-and-pepper noise, the study demonstrates the critical role of memory management and vectorization in high-performance computing.

Key Findings & Performance Analysis

The execution results from main.py reveal a significant hierarchy in processing speeds:

- Pure Python Baseline: This method serves as the unoptimized control, relying on nested loops that process pixels individually, resulting in the highest execution times across all filters.
- NumPy Optimization: By shifting to vectorized operations, the Sobel filter achieved a massive 72.9x speedup, reducing execution from 0.1110s to just 0.0015s.
- Cython-Style Efficiency: The use of contiguous memory buffers and integer arithmetic allowed the Median filter to reach its peak efficiency at 0.0017s, outperforming the baseline by over 30 times.

Visual Effectiveness

Beyond speed, the filters successfully achieved their mathematical objectives:

- The Median filter proved most effective for this specific test case, completely removing salt-and-pepper noise while maintaining sharp object edges.
- The Sobel filter clearly isolated the boundaries of the square and circle, proving the accuracy of the gradient magnitude calculations.
- The Gaussian filter provided uniform smoothing, effectively reducing high-frequency variance in the background gradient.

The results underscore that while Pure Python is sufficient for logic verification, NumPy and Cython-style optimizations are indispensable for real-world data science applications where execution time and resource management are paramount.